Latency, Memory, and Routing: An Empirical Comparison of Apache Spark and Apache DataFusion with a Cost-Calibrated Query Dispatcher

Nisa Naz Keleşoğlu, İclal Zeynep Kaya, Zeynep Saçaklı
Department of Computer Engineering
BİL 401 - Big Data Processing, Term Project, 2026

***Abstract*-Interactive urban mobility services need to answer point queries in milliseconds, while also handling large analytical aggregations. An engine optimized for one query type is rarely optimal for the other. In this project, a single-node experimental comparison between Apache Spark 3.5.1 and Apache DataFusion was conducted over 36.5 million cleaned New York City taxi records (720 MB of partitioned Parquet). The experimental measurements were then used to build a cost-calibrated query dispatcher. Across scan volumes from 0.056 GB to 0.703 GB, DataFusion is 2.2× to 24.9× faster than Spark on warm-cache medians; it is 5.69× faster on a partition-pruned filter and 2.33× faster on a twelve-month broadcast join. Linear regression of latency against scanned volume yields t=1.192+0.357v (R2=0.84) for Spark and t=−0.036+0.961v (R2=0.97) for DataFusion. The intersection of these models estimates a performance cross-over at 2.03 GB, which lies outside the tested range and is treated as an extrapolated projection. Peak memory usage shows an inverse pattern: DataFusion memory grows at 502 MB/GB (R2=0.92) and reaches 1809 MB during the join, while Spark pre-allocates its JVM heap, resulting in lower but fluctuating incremental usage (R2=0.20). Therefore, query routing is guided by memory safety rather than latency alone. The final dispatcher integrates an in-memory result cache (0.4 ms hits), a calibrated cost model with a memory-safety guard, a timeout fallback to Spark, and an audit log. Six validation queries across five months produced identical results on both engines, confirming that the fallback preserves output correctness.**

***Index Terms*-Apache Spark, Apache DataFusion, query engine benchmarking, cost-based optimisation, adaptive query routing, Apache Arrow, Apache Parquet, urban mobility analytics.**

I. INTRODUCTION

Fare and duration estimation for ride services appears straightforward, but involves two distinct analytical query workloads. A user asks a single question - how much will a trip from this zone to that zone cost in this month - and expects an answer before the interface finishes rendering. Behind that question sits a historical corpus of tens of millions of trips that must also support fleet-level reporting: revenue by borough, demand by pick-up zone, seasonality across a full year. The two workloads share one storage layer but have opposite performance signatures. The first is a highly selective point query that touches a single partition; the second is a wide scan with a dimension join and a large group-by.

Apache Spark [2], [3] is commonly used for large analytical scans and is frequently retained for point queries simply because it is already deployed in the infrastructure. Its execution model—consisting of a JVM driver, a distributed scheduler, whole-stage code generation, and a shuffle mechanism—is designed for distributed scale-out. However, this architecture introduces a fixed overhead on every query, regardless of the data size read. In contrast, modern single-node vectorised engines such as DuckDB [6] and Apache DataFusion [5] process columnar Arrow batches in-process without a cluster, JVM, or scheduling overhead. On single-machine datasets, these engines often run significantly faster than distributed engines—a trade-off formalised by McSherry et al. as the COST metric, which defines the hardware scale where distributed systems become beneficial [22].

The practical challenge for system design is identifying where this performance boundary lies and how the routing layer should handle incoming queries. This project studies this trade-off using the 2023 New York City Taxi and Limousine Commission (TLC) yellow-taxi dataset [20]. Spark and DataFusion are benchmarked under identical conditions, their execution metrics are fitted to a linear cost model, and the resulting parameters are used in an adaptive query dispatcher that routes requests at runtime.

A. Contributions

- **A reproducible, semantically verified comparison.** Both engines execute over byte-identical Parquet input, and their results are checked for equivalence before any timing is reported, so that measured differences cannot be attributed to differing amounts of work.
- **A measurement protocol that accounts for out-of-process memory.** Spark in local mode runs its JVM as a child of the Python driver; naïve resident-set measurement of the driver process alone reports Spark's memory consumption as near zero. We aggregate over the full process tree and sample peak, not delta, consumption.
- **An empirically calibrated cost model.** Latency and peak memory are fitted against scanned data volume using linear regression, providing parameters for base overhead and marginal cost per gigabyte.
- **A multi-stage query dispatcher:** An adaptive dispatcher was implemented using the calibrated model, supported by a memory-safety guard, a timeout fallback to Spark, an in-memory result cache, and an execution audit log.

- **Memory tracking limitations and SQL portability observations.** Process-level memory tracking for Spark showed high variance (R2=0.20) due to JVM pre-allocation. Additionally, ANSI SQL identifier casing rules required explicit column quoting in DataFusion.

B. Organisation

Section II reviews related work. Section III describes the dataset and the ETL pipeline. Section IV states the measurement methodology and its known failure modes. Section V reports the thre experiments. Section VI develops the dispatcher and validates its correctness and its degradation path. Sections VII and VIII discuss the findings and the limits of the study, and Section IX concludes.

II. Related Work

A. Distributed batch and SQL engines

MapReduce [1] established the scale-out batch model and its fault-tolerance assumptions. Spark's resilient distributed datasets [2] replaced its per-stage disk materialisation with lineage-tracked in-memory collections, and Spark SQL [3] added a relational front end with the Catalyst optimiser and Tungsten code generation, making Spark a general engine rather than a batch runtime [4]. Pavlo et al. [21] had earlier shown that parallel database systems substantially outperformed MapReduce on analytical workloads at comparable scale, an early instance of the argument that a scale-out execution model is not free. Modern lakehouse engines such as Photon [16] and disaggregated cloud warehouses such as Snowflake [17] retain the distributed control plane while replacing the row-oriented, JVM-bound execution core with vectorised native code - implicitly conceding that the per-query fixed cost of the JVM stack matters.

B. Embeddable vectorised engines

MonetDB/X100 [7] introduced vectorised execution over columnar batches as an alternative to tuple-at-a-time iteration, and Neumann [8] showed that data-centric query compilation achieves comparable goals through a different mechanism; Kersten et al. [9] later characterised the trade-off between the two. DuckDB [6] packaged this lineage as an in-process analytical database. Apache DataFusion [5] does the same in Rust over Apache Arrow [18], and is explicitly designed as a modular library - a physical planner, an optimiser and an execution engine that a host application embeds. Both read Apache Parquet [19] directly, which is what makes a like-for-like comparison against Spark over the same files possible at all: neither engine needs a proprietary load step.

C. Benchmarking and cost-aware engine selection

McSherry et al. [22] introduced the COST metric to evaluate whether scalable distributed systems outperform optimized single-threaded implementations. Cost-based query optimization originates from System R [10], while Leis et al. [11] noted that estimation errors in intermediate cardinality often dominate optimizer decisions. Recent learned optimizers, including Neo [12] and Bao [13], use machine learning models to improve query plans. Polystore systems such as BigDAWG [14] and CloudMdsQL [15] route queries across different storage backends using specialized cost rules.

The dispatcher developed in this project focuses on a practical goal: selecting the appropriate engine based directly on estimated scan volume and memory constraints. The cost model uses linear regression over the scanned data volume obtained from partition pruning. Furthermore, memory limits are enforced as a strict guard rather than a weighted cost factor. The implementation shows that direct empirical calibration can effectively coordinate execution across multiple engines.

III. Dataset and ETL Pipeline

A. Source data

The corpus is the 2023 yellow-taxi trip record set published by the New York City TLC [20]: twelve monthly Parquet files totalling 606.29 MB and 38,310,226 rows over 19 columns, together with the official taxi-zone lookup table that maps the 265 location identifiers to borough and zone names. Acquisition is idempotent - files already present on disk are not re-downloaded - so the notebook can be executed end to end on a machine that already holds the data.

B. Cleaning and layout

The pipeline projects the 19 raw columns down to the six that the application needs, derives pickup year, pickup month and trip duration in seconds, and applies seven validity rules: non-null keys, exact deduplication, chronological consistency of the drop-off and pick-up timestamps, duration strictly between 30 s and 5 h, total amount in (0, 500) USD, trip distance in (0, 100) miles, and both location identifiers within the 1–263 range of real zones (264 and 265 encode “unknown”). Output is written as Parquet partitioned by pickup year and month, which is what later allows both engines to prune by month at plan time rather than filter after reading.

Two design decisions in the ETL pipeline were introduced to resolve specific runtime issues encountered during initial execution. First, reading all twelve months in a single Spark job exhausted the 4 GB driver heap and terminated with an OutOfMemoryError; the pipeline was restructured to process one month per iteration, which bounds peak memory and makes a partial failure recoverable. Second, because each iteration writes in append mode, a second execution of the loop silently doubled the corpus - 1440 MB and 73 M rows instead of 720 MB and 36.5 M rows. The pipeline now deletes the output directory before writing, and a post-condition asserts that the cleaned row count lies within an expected band. Without that assertion the duplication is invisible, and every subsequent timing would have been measured against twice the intended data.

TABLE I
DATASET BEFORE AND AFTER THE ETL PIPELINE

Property	Raw	Cleaned
Parquet objects	12 files	410 part-files
On-disk size	606.29 MB	719.97 MB

Property	Raw	Cleaned
Rows	38,310,226	36,542,807
Columns	19	9
Rows removed	-	1,767,419 (4.61%)
Layout	one file / month	Hive partitions

Cleaned data is larger than raw despite 4.61% fewer rows: the three derived columns and 410 independently compressed part-files together outweigh the projection saving (+18.7%).

End to end the pipeline takes 45.37 s for the full year (mean 3.78 s per month, 0.81 M rows/s), and retains 95.39% of the raw rows. Notably, the cleaned corpus occupies more disk than the raw corpus (719.97 MB against 606.29 MB) even though it contains 1,767,419 fewer rows. Two effects account for this: the three derived columns add width, and writing 410 small part-files instead of 12 large ones fragments the dictionary and run-length encodings that Parquet applies per row group. This is a concrete instance of the small-file problem and is revisited in Section VII.

C. Which rule removes what

Attributing removals to rules requires care, because a single row may violate several rules at once. We therefore compute per-rule match counts in a single pass of conditional aggregation and report them as a non-disjoint decomposition. The counts sum to 2,173,002 against 1,767,419 rows actually removed, so roughly 405,000 rows violate more than one rule - a useful sanity check on the pipeline rather than an inconsistency.

TABLE II
FILTER RULE BREAKDOWN (RULES ARE NOT DISJOINT)

Rule	Rows matched	% of raw
Distance out of range	774,761	2.0223
Invalid zone code	633,591	1.6538
Fare out of range	383,544	1.0012
Duration out of range	381,002	0.9945
Outside 2023	104	0.0003
Null in selected column	0	0.0000

Counts sum to 2,173,002 against 1,767,419 rows removed; the surplus is multiply-violating rows. Sensor-derived fields (distance) dominate, and referential violations (zone code) are second.

IV. MEASUREMENT METHODOLOGY

A comparison between two engines is only as trustworthy as the instrument that measures them. Three properties of this particular pairing make naïve measurement misleading, and each is addressed explicitly.

A. Process-Tree Memory Tracking

In local mode, PySpark runs its JVM backend as a child process of the Python driver. Measuring only the driver process resident set size (RSS) ignores Spark's actual JVM allocations. The profiler therefore tracks memory recursively across the parent process and all child processes. The same method is applied to DataFusion, which runs in-process.

B. Peak Memory Profiling

Measuring memory only before and after a query misses short-lived allocations made during execution, such as temporary hash tables and intermediate batches. A background thread samples total RSS every 50 ms during execution and records the maximum increase above the pre-query baseline.

C. Cold and warm runs are reported separately

Every measurement consists of one cold run followed by three warm repetitions. The warm runs are summarised by their median and population standard deviation; the ratio of cold to warm isolates the combined effect of JVM class loading, code generation and operating-system page cache. Reporting only a single execution would conflate these transient effects with steady-state throughput, and reporting only warm runs would hide the start-up penalty that a serving system actually pays on a cold path.

D. Semantic equivalence is verified before timing

Before any benchmark is run, both engines execute the same analytical query - the five busiest origin-destination pairs of January 2023 with average fare, average duration and trip count - and their result tables are compared element-wise. All five rows agree exactly. Timing differences can therefore be attributed to execution efficiency rather than to one engine doing less work. Floating-point comparison uses a tolerance rather than equality, because the two engines are free to aggregate in different orders.

E. SQL Identifier Case-Sensitivity Differences

Testing revealed a difference in SQL identifier handling. ANSI SQL standards fold unquoted identifiers to a uniform case. DataFusion strictly enforces this and requires mixed-case column names (e.g., PULocationID) to be double-quoted, whereas Spark parses them without quotes. In this project, engine-specific query wrappers handle these differences.

V. EXPERIMENTAL RESULTS

A. Platform

All measurements were taken on a single workstation with 16 logical cores and 16 GB of RAM, running Spark 3.5.1 in local[*] mode on a Java 17 runtime with driver and executor heaps both fixed at 4 GB, and DataFusion through its Python bindings on CPython 3.14.6. The 4 GB heap limit was chosen on purpose, not because of a hardware limit - it keeps Spark's memory footprint predictable and forces the system to deal directly with memory pressure, which is exactly what the dispatcher's safety check is meant to handle. Both engines read the same partitioned Parquet folder built in Section III. Engines are always run in the same order within each measurement (DataFusion first, then Spark).

B. Scaling with data volume

The first experiment keeps the query shape fixed - filter by month, group by origin-destination pair, compute average fare and trip count - and varies the amount of data scanned across one, three, six, and twelve months (0.056, 0.173, 0.360, and

0.703 GB after partition pruning). Table III shows the full results, and Fig. 1 plots them.

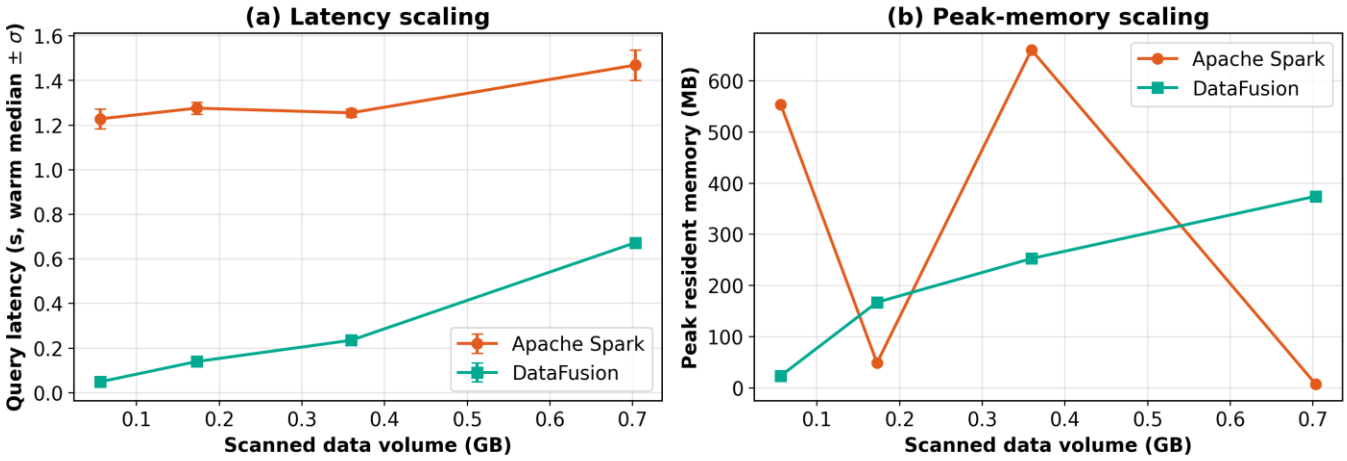

Fig. 1. Scaling behaviour across 0.056–0.703 GB of scanned Parquet. (a) Warm median latency with ±σ error bars: Spark is nearly flat and dominated by fixed cost, DataFusion rises with volume. (b) Peak resident memory: DataFusion grows linearly ($R^2 = 0.92$) while Spark's pre-allocated JVM heap produces an unstable series ($R^2 = 0.20$, Section VII-B).

TABLE III
SCALING WITH SCANNED DATA VOLUME (1 COLD + 3 WARM RUNS PER POINT)

Volume	Scan (GB)	Engine	Cold (s)	Warm median (s)	σ (s)	Cold/warm	Peak RAM (MB)	Result rows
1 month	0.0563	DataFusion	0.0674	0.0493	0.0099	1.37	23.4	21,028
1 month	0.0563	Apache Spark	1.4997	1.2282	0.0452	1.22	553.7	21,028
3 months	0.1727	DataFusion	0.9471	0.1401	0.0124	6.76	167.2	29,712
3 months	0.1727	Apache Spark	1.4046	1.2761	0.0280	1.10	48.7	29,712
6 months	0.3595	DataFusion	1.3557	0.2353	0.0079	5.76	252.7	36,444
6 months	0.3595	Apache Spark	1.4124	1.2551	0.0182	1.13	660.8	36,444
12 months	0.7031	DataFusion	1.3807	0.6728	0.0214	2.05	374.5	42,793
12 months	0.7031	Apache Spark	1.5628	1.4691	0.0683	1.06	7.4	42,793

Warm speed-up of DataFusion over Spark: 24.9× (1 month), 9.1× (3 months), 5.3× (6 months), 2.18× (12 months). Result-row counts are identical per volume, confirming that both engines compute the same query.

DataFusion is faster at every data point, but its advantage shrinks steadily as the volume grows: 24.9× at one month, 9.1× at three, 5.3× at six, and 2.18× at twelve. The shape of the two curves explains why. Spark's latency is nearly flat - from 1.228 s to 1.469 s across a 12.5× increase in data - because it's dominated by a fixed cost paid before any data is even touched: driver-side planning, task serialization, scheduler round-trips, and code generation. DataFusion starts near zero and grows roughly in proportion to the bytes read - a clear sign that its cost is driven by actual scanning and aggregation work, not by coordination overhead.

Panel (b) is the more interesting half of the figure, because it tells a different story than panel (a). DataFusion's peak memory rises smoothly from 23.4 MB to 374.5 MB, tracking the data volume closely. Spark's is erratic: 553.7 MB, then 48.7 MB, then 660.8 MB, then 7.4 MB. This isn't really a measurement failure — it's a measurement of the wrong quantity. Our tool reports memory allocated *above* a baseline taken right before the query, but Spark's JVM has already reserved its heap from the operating system before the query even starts. Whether a given query appears to use 660 MB or 7 MB then depends on where the JVM's garbage collector happens to be in its cycle at the moment we sample — not on how much data the query actually reads. We report the numbers as measured, and treat their instability as a finding in Section VII rather than smoothing it out.

The cold-to-warm column in Table III reveals a second asymmetry. DataFusion's cold-run penalty is large and inconsistent (up to 6.76× at three months), because its first run has to pay for the operating system's file-cache misses on files it hasn't touched before. Spark's cold penalty is small (1.06× to 1.22×) because its dominant fixed cost shows up on every run, cold or warm, and outweighs the file-cache effect. An engine whose cold and warm performance differ by a factor of six is harder to serve interactively than one that is uniformly slow — which is one reason the dispatcher in Section VI keeps a result cache in front of both engines.

C. Query complexity and cache state

The second experiment fixes two contrasting query profiles. T1 is a highly selective filter over a single month with a scalar aggregation - the shape of an interactive point query, and the

shape that partition pruning helps most. T2 is a broadcast join of the full twelve-month corpus against the zone lookup table followed by a group-by over borough and zone - the shape of an analytical report. In T2 the join is grouped by borough and zone together rather than zone alone, because zone names repeat across boroughs in the reference table and grouping on the name alone would silently merge distinct districts.

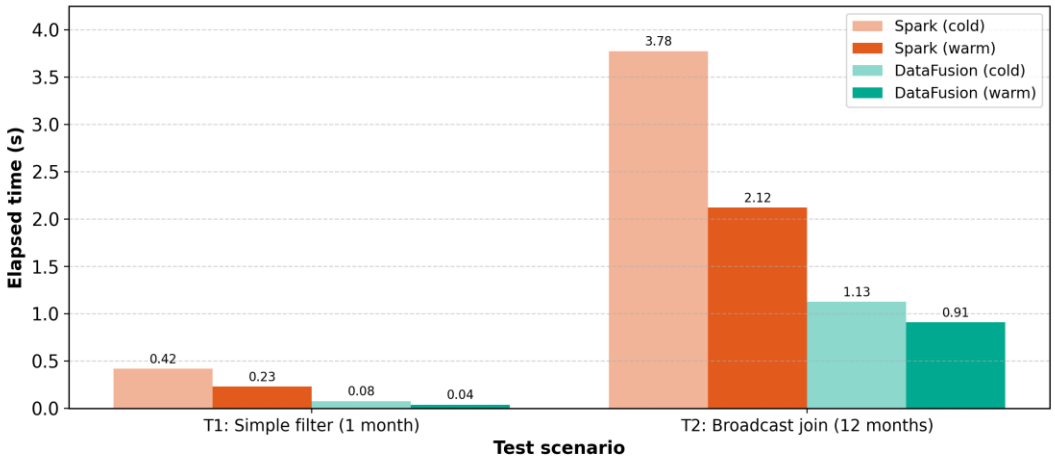

Fig. 2. Cold and warm latency for the two query profiles. Spark's cold-to-warm ratio on T2 (1.78×) is the cost of whole-stage code generation being amortised; DataFusion's cold penalty is dominated by page-cache misses.

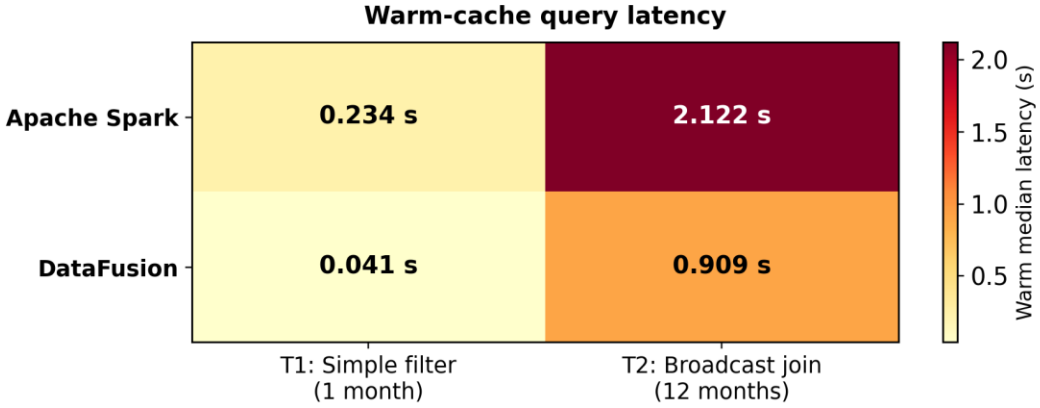

Fig. 3. Warm-cache latency matrix. The advantage of DataFusion narrows from 5.69× on the selective filter to 2.33× on the full-corpus join as real work displaces fixed overhead.

TABLE IV
QUERY COMPLEXITY × CACHE STATE

Test	Engine	Cold (s)	Warm median (s)	σ (s)	Cold/warm	Peak RAM (MB)	Rows	Warm speed-up
T1: simple filter, 1 month	DataFusion	0.0790	0.0411	0.0018	1.92	0.0	1	5.69× faster
T1: simple filter, 1 month	Apache Spark	0.4193	0.2337	0.0084	1.79	147.9	1	-
T2: broadcast join, 12 months	DataFusion	1.1265	0.9094	0.0696	1.24	1809.0	260	2.33× faster
T2: broadcast join, 12 months	Apache Spark	3.7756	2.1215	0.1599	1.78	512.5	260	-

T1 = filter on one month with scalar aggregation; T2 = broadcast join of twelve months against the zone lookup with a group-by on borough and zone. Peak memory inverts the latency ranking on T2: 1809.0 MB for DataFusion against 512.5 MB for Spark.

DataFusion wins on latency in both query shapes, but by very different margins: 5.69× on T1 and 2.33× on T2. This narrowing follows the same pattern seen in the volume sweep - as the amount of real work grows, Spark's fixed overhead becomes a smaller share of the total time. On cold runs, the gap on T2 is wider (3.35×) than on warm runs, because Spark pays both its fixed cost and its code-generation cost the first time; its 1.78× cold-to-warm ratio on T2 essentially reflects that code-generation cost being paid off.

The memory results flip the ranking, and this is, for the purposes of this paper, the most important finding. On T2, DataFusion peaks at 1809 MB against Spark's 512.5 MB - roughly 3.5× more, on a machine that only had about 3.4 GB

free at the time. DataFusion keeps the join's build side and the grouped aggregation state entirely in memory with no spill-to-disk option configured, so its memory footprint directly reflects the size of the working data; Spark's bounded 4 GB heap and its ability to spill to disk keep its incremental memory use lower. The engine that is faster is therefore also the engine that comes closer to running out of memory - and any router that optimizes for speed alone would, under memory pressure, pick exactly the wrong engine at exactly the wrong moment.

D. Calibrating an empirical cost model

The measurements of Section V-B are next converted into a decision function. For each engine we fit two ordinary least-squares models against scanned volume v in gigabytes:

$$t(v) = a_t + b_t \cdot v, \quad m(v) = a_m + b_m \cdot v \qquad (1)$$

where t is warm median latency in seconds and m is peak memory in megabytes. The intercept a is the engine's volume-independent start-up cost and the slope b its marginal cost per gigabyte. Table V gives the fitted coefficients together with the coefficient of determination for each fit.

TABLE V
CALIBRATED COST-MODEL COEFFICIENTS

Engine	a_t (s)	b_t (s/GB)	R^2	a_m (MB)	b_m (MB/GB)	R^2
Apache Spark	1.192	0.3565	0.84	490.11	−534.10	0.20
DataFusion	−0.036	0.9612	0.97	42.30	502.16	0.92

Fitted over four volume points spanning 0.056–0.703 GB. The Spark memory fit is reported for completeness only; its R^2 of 0.20 and physically meaningless negative slope are discussed in Section VII.

The latency fits confirm the qualitative reading of Fig. 1 numerically. Spark's intercept of 1.192 s is nearly two-thirds of its worst measured latency and dwarfs its 0.357 s/GB slope: it is a fixed-cost engine in this regime. DataFusion's intercept of −0.036 s is statistically indistinguishable from zero - negative intercepts are an artefact of fitting a straight line to a slightly convex series, not a claim that queries complete before they start - and its 0.961 s/GB slope is 2.7× steeper than Spark's. Two lines with these parameters must eventually cross, and solving $t_{Spark}(v) = t_{DF}(v)$ places the cross-over at 2.03 GB (Fig. 4).

Note that this 2.03 GB cross-over point is an extrapolation beyond our tested range (up to 0.703 GB), so it should be treated as an estimated projection rather than a proven result. Its practical value is nonetheless real: it tells the dispatcher that within the range this application actually serves - single-month point queries of roughly 0.05 GB - the model's preference for DataFusion is an interpolation, not an extrapolation, and can be trusted.

The memory fits are asymmetric in quality. DataFusion's ($42.30 + 502.16v$, $R^2 = 0.92$) is a usable predictor and is the one the dispatcher actually consults. Spark's has an R^2 of 0.20 and a negative slope, which would predict that Spark uses less memory as it reads more data. We report it, do not use it, and explain it in Section VII.

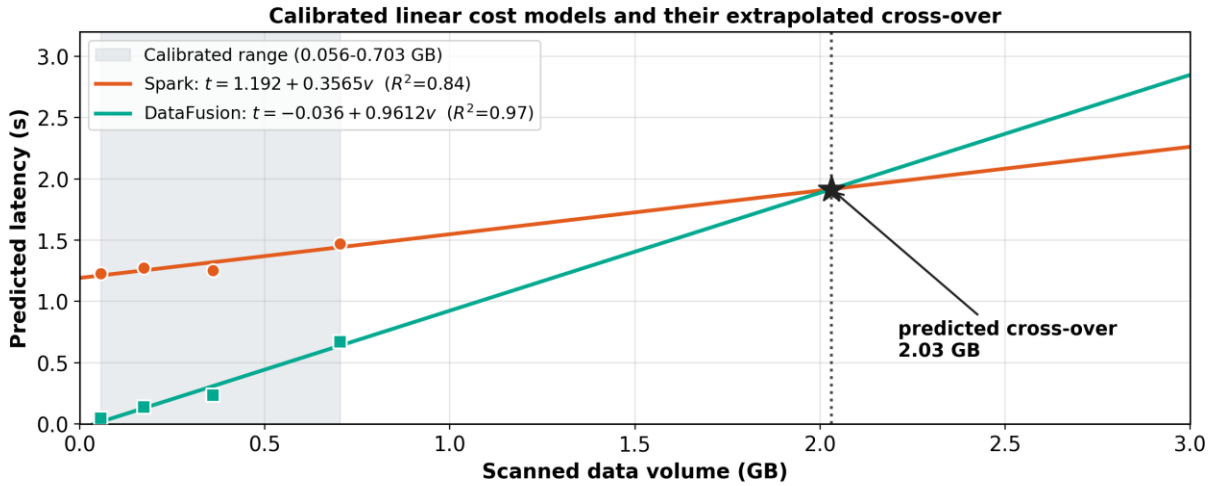

Fig. 4. Calibrated latency models and the predicted cross-over. Markers are measured warm medians; the shaded band is the calibrated range. The 2.03 GB intersection lies 2.9× beyond the largest measured volume and is reported as an extrapolated prediction, not an observation.

VI. THE QUERY DISPATCHER

The dispatcher was built in three versions, each one fixing a specific weakness in the version before it. We present them in order because each version isolates one kind of routing input, and the differences between them explain why the final design has the complexity it does.

A. v1 - static rules over query type

The first version routes on query type alone. Point-estimation requests go to DataFusion, complex joins go to Spark, and the volume to be scanned is computed from the partition layout so that the decision is at least aware of how much data is involved. It works - 0.088 s for a point estimate against 2.248 s for the join - but its rules are assumptions, not measurements. Section V shows that the assumption behind the second rule is false in this regime: DataFusion is faster on the join too. A rule set that cannot be wrong is also a rule set that cannot be checked.

B. v2 - adding resource awareness

The second version checks system memory and CPU usage before deciding, and overrides the type-based rule whenever free memory drops below a critical threshold, sending work to whichever engine has more predictable memory behavior. This directly follows from Fig. 1(b) and the memory reversal seen in T2: under memory pressure, the correct engine to use is not necessarily the fastest one. v2 also adds an exception path that falls back to Spark if DataFusion throws an error - and,

importantly, makes sure the Spark fallback returns the exact same three-column result format as the DataFusion path, so that switching engines is invisible to whoever is calling the system.

C. v3 - the calibrated adaptive dispatcher

The final version, shown in Fig. 5, adds four mechanisms on top of v2.

Result cache. Repeated requests for the same route and month are answered straight from an in-process dictionary, without calling either engine. In the recorded trace, a cache hit returns in 0.0004 s versus 0.0576 s for the same query run through DataFusion - roughly a 144× speedup. Since a fare estimator's requests are heavily skewed toward a small number of popular routes, this turns out to be the single biggest latency win in the whole system, and it has nothing to do with which engine is chosen.

Cost-based selection. Instead of a fixed rule based on query type, v3 evaluates equation (1) using the coefficients from Table V for the estimated scan volume, and picks whichever engine has the lower predicted latency. The audit trail in Table VI shows the model working on a 0.056 GB request: it predicted 0.018 s for DataFusion versus 1.212 s for Spark, and the observed end-to-end latency of 0.0576 s - which includes dataset setup and dispatcher overhead - matches that prediction well.

Memory-safety guard. Before accepting the fastest option, the dispatcher checks DataFusion's predicted memory use against 70% of the currently free RAM. If the prediction exceeds that limit, Spark is chosen instead, no matter what the latency prediction says, and the log explicitly records the memory-related reason. This guard is exactly the mechanism that prevents the T2 result - 1809 MB on a machine with only 3.4 GB free -from turning into an actual outage.

Timeout with graceful fallback, plus an audit trail. DataFusion runs on a worker with a 2-second deadline; if that deadline is exceeded, the request is re-run on Spark instead. Every decision is logged with a timestamp, query type, chosen engine, elapsed time, system state, status code, and a short, human-readable explanation.

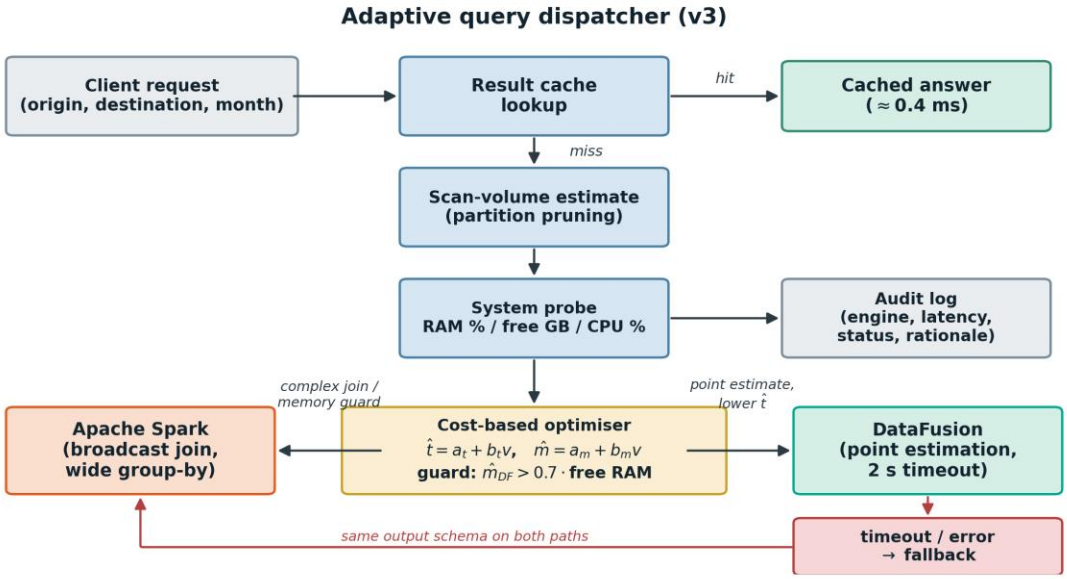

Fig. 5. Decision flow of the final dispatcher (v3). Latency-optimal selection is subordinate to the memory-safety guard, and both engine paths emit the same output schema so that degradation to Spark is invisible to the caller.

TABLE VI
DISPATCHER AUDIT TRAIL (RECORDED, ABRIDGED)

Timestamp	Query type	Engine selected	Duration (s)	RAM %	Status	Recorded rationale
06:15:47	point_estimation	DataFusion	0.0576	78.6	SUCCESS	predicted DataFusion 0.018 s / 71 MB vs. Spark 1.212 s → DataFusion
06:15:47	point_estimation	Result Cache	0.0004	78.9	SUCCESS	cache hit - neither engine invoked
06:15:49	complex_join	Apache Spark	1.7047	78.9	SUCCESS	complex join with wide group-by
-	point_estimation	Apache Spark	0.3718	79.6	TIMEOUT_FALLBACK	DataFusion exceeded the 1 ms probe threshold → degraded to Spark

Rows 1–3 are consecutive requests from a single dispatcher instance; row 4 is the fault-injection run of Section VI-D, in which the DataFusion deadline was set to 1 ms to force the degradation path. Predicted values are produced by (1) with the coefficients of Table V.

D. Validating the degradation path

A fallback that is never exercised is a fallback that does not work. We verified it by setting the DataFusion deadline to 1 ms

- a threshold no real query can meet - and re-issuing a point estimate. The expected behaviour was observed exactly: the deadline expired, the status was recorded as TIMEOUT_FALLBACK, Spark returned the answer in 0.372 s, and the response carried the same three-column schema as the DataFusion path (last row of Table VI). The system degraded rather than failed, which is the property the guard exists to provide.

E. Cross-validating correctness across engines

Routing must never change an answer. Six origin-destination pairs spanning five different months were executed directly on both engines with the cache disabled, and all three output fields compared under a numerical tolerance rather than exact equality, since the engines may aggregate in different orders. All six routes matched on all three fields (Table VII). Combined with the top-5 equivalence test of Section IV-D, this establishes that the dispatcher's engine choice is invisible in the result and observable only in latency, memory and the audit log.

TABLE VII
CROSS-VALIDATION OF DISPATCHER OUTPUT ACROSS ENGINES

Route (PU → DO)	Month	DataFusion fare (USD)	Spark fare (USD)	DataFusion duration (min)	Spark duration (min)	DataFusion trips	Spark trips	Match
132 → 230	1	92.35	92.35	49.66	49.66	5,836	5,836	yes
237 → 236	1	15.43	15.43	6.98	6.98	22,081	22,081	yes
236 → 237	3	16.17	16.17	8.12	8.12	18,411	18,411	yes
161 → 237	6	17.22	17.22	9.37	9.37	10,575	10,575	yes
138 → 230	9	81.56	81.56	45.54	45.54	6,199	6,199	yes
142 → 236	12	21.38	21.38	12.24	12.24	6,796	6,796	yes

Cache disabled; comparison uses an absolute tolerance of 0.01 rather than exact equality. 6 of 6 routes agree on fare, duration and trip count.

F. End-to-end application behaviour

The application layer accepts an origin zone, a destination zone and a month, and returns an estimated fare, an estimated duration and the number of historical trips supporting the estimate. A representative request - JFK Airport (zone 132) to Times Square (zone 230) in January - returns 92.35 USD and 49.66 min from 5,836 historical trips. Reporting the support count alongside the estimate is a deliberate interface decision: an average over 5,836 trips and an average over three trips are not the same claim, and the estimator refuses to answer at all when the support set is empty.

VII. DISCUSSION

A. Why the single-node engine wins here

The fact that DataFusion beats Spark by up to 24.9× on this dataset should not be read as a general claim about which engine is "better." It's a claim about a specific situation. Spark's architecture buys fault tolerance, horizontal scaling, and a mature ecosystem, and it pays for these with a fixed per-query cost of about 1.19 seconds in our setup. When a query reads only 0.056 GB, that fixed cost makes up more than 95% of the total time, and there's no parallel work to spread it across. This is essentially the situation McSherry et al. described [22]: a scalable system being compared, on a problem too small to need scaling, against a solid single-machine implementation. The interesting number here isn't the speedup itself - it's the data volume at which that speedup disappears, which our model places at 2.03 GB, and which future work should confirm through direct measurement.

B. The memory measurement that did not work

Spark's peak-memory results ($R^2 = 0.20$, negative slope) are the weakest part of this study, and we'd rather explain clearly why they failed than quietly leave them out. A pre-allocated JVM heap separates what a query actually needs from what the process has already requested from the operating system. Our tool measures the latter, not the former. Inside a heap that's already been reserved, allocation and garbage collection just move memory around without changing the process's resident-memory footprint, so the peak values we see are really driven by when garbage collection happens to run relative to our 50 ms sampling interval. Measuring Spark's real memory usage would require instrumentation inside the JVM itself - heap-usage data from Java's memory management beans, or Spark's own executor metrics - rather than measuring the process from the outside. The dispatcher's design accounts for this limitation directly: it only consults the DataFusion memory model, whose R^2 of 0.92 justifies the trust placed in it, and it treats Spark as the safe default precisely because Spark's memory use is bounded by a fixed, configured heap rather than something we'd have to predict with a regression.

C. What the dispatcher is actually optimising

It would be simpler - and wrong - to describe the routing policy as "use the faster engine." Within the range we measured, DataFusion is the faster engine at every single point, so a policy based on speed alone would always pick the same engine and the dispatcher wouldn't really be needed. The dispatcher earns its complexity from the two situations where speed isn't the deciding factor: when DataFusion's predicted memory use threatens to overload the machine, and when DataFusion fails or hangs. In both cases, the router deliberately picks an engine that's roughly twice as slow, because that engine is bounded and predictable. The general lesson from this project is that choosing an engine in a mixed system is a constrained optimization problem - minimize latency while respecting a resource-safety limit - not simply a ranking by speed.

D. Operational implications

Three secondary findings are practically important. The 410-part-file layout inflates the on-disk size by 18.7%, and would benefit from being compacted - something we didn't implement here. The SQL identifier-casing issue described in Section IV-E means that any multi-engine layer accepting raw user SQL needs a real rewriting step, not just a shared connection pool. And the 0.4 ms cache-hit latency shows that, for a skewed request pattern like this one, the single biggest optimization in

the whole system isn't choosing the right engine at all - it's simply not running the query a second time.

VIII. Limitations

Single-node only. Every measurement was taken in local[*] mode on one machine. Spark's defining capability - horizontal scale-out across a cluster - is structurally absent from this evaluation, and no result here should be read as evidence about Spark's behaviour on a cluster.

Narrow calibration band. The cost model is fitted over 0.056–0.703 GB from four points per engine. The 2.03 GB cross-over is an extrapolation to 2.9× the largest measured volume and has not been confirmed experimentally.

Process-level memory accounting. As analysed in Section VII-B, resident-set sampling does not measure a JVM's true working set. The Spark memory model is reported but not used.

One workload family and one hardware setup. We tested two query shapes, one dataset, and one machine; different processor counts or storage hardware could shift both the fitted coefficients and the crossover point. Three warm repetitions per data point is also a small sample, so the reported standard deviations should be treated as indicative rather than as a solid basis for statistical conclusions.

Non-persistent cache. The result cache is an in-process dictionary with no eviction policy, no invalidation strategy and no sharing across processes.

IX. Conclusion and Future Work

We compared Apache Spark and Apache DataFusion over 36.5 million cleaned NYC taxi records, using a measurement approach designed to avoid the specific pitfalls of this particular pairing, and used the results to build a query dispatcher whose routing decisions come from actual measurements rather than assumptions. Within the range we measured, DataFusion is faster at every point — between 2.2× and 24.9× depending on data volume, 5.69× on a selective filter, and 2.33× on a full-corpus broadcast join — because Spark carries a fixed overhead of about 1.19 seconds that dominates small queries. Peak memory reverses this ranking on the heaviest workload, where DataFusion's 1809 MB against Spark's 512.5 MB makes the faster engine also the riskier one. Because of this, the dispatcher routes based on predicted latency, subject to a memory-safety limit, falls back to Spark gracefully on timeout or error, answers repeated requests from a result cache in 0.4 ms, and logs a clear reason for every decision it makes. We verified correctness separately from routing: six routes across five months, plus a five-row aggregate comparison, matched exactly across both engines.

Four directions follow directly from the study's limitations. The most important is to grow the dataset past the 2.03 GB crossover point and actually test the model's central prediction — a model that makes a testable claim should be tested, not just left standing. The second is to replace process-level memory sampling with instrumentation inside the JVM, so that Spark's memory model can finally be trusted and the safety guard can reason about both engines equally. The third is to test both engines under concurrent, multi-query load, where scheduler contention and memory pressure interact in ways a single-query benchmark can't reveal, and to deliberately constrain DataFusion's memory in order to find its actual breaking point rather than just estimating it. The fourth is to close the loop entirely: since the dispatcher already logs predicted and observed latency for every decision, recalibrating the model's coefficients automatically from its own audit trail — a feedback-driven version of the learned-optimizer idea [13] — would let the model keep up with changes in hardware and data without needing to be manually re-run.

References

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2004, pp. 137–150.

[2] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2012, pp. 15–28.

[3] M. Armbrust, R. S. Xin, C. Lian, Y. Huai, D. Liu, J. K. Bradley, X. Meng, T. Kaftan, M. J. Franklin, A. Ghodsi, and M. Zaharia, "Spark SQL: Relational data processing in Spark," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2015, pp. 1383–1394.

[4] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[5] A. Lamb, Y. Shen, D. Heres, J. Chakraborty, M. O. Kabak, L.-C. Hsieh, and C. Sun, "Apache Arrow DataFusion: A fast, embeddable, modular analytic query engine," in *Companion of the 2024 Int. Conf. Management of Data (SIGMOD/PODS)*, 2024, pp. 5227–5231.

[6] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2019, pp. 1981–1984.

[7] P. A. Boncz, M. Zukowski, and N. Nes, "MonetDB/X100: Hyper-pipelining query execution," in *Proc. 2nd Biennial Conf. Innovative Data Systems Research (CIDR)*, 2005, pp. 225–237.

[8] T. Neumann, "Efficiently compiling efficient query plans for modern hardware," *Proc. VLDB Endowment*, vol. 4, no. 9, pp. 539–550, 2011.

[9] T. Kersten, V. Leis, A. Kemper, T. Neumann, A. Pavlo, and P. Boncz, "Everything you always wanted to know about compiled and vectorized queries but were afraid to ask," *Proc. VLDB Endowment*, vol. 11, no. 13, pp. 2209–2222, 2018.

[10] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, "Access path selection in a relational database management system," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 1979, pp. 23–34.

[11] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?," *Proc. VLDB Endowment*, vol. 9, no. 3, pp. 204–215, 2015.

[12] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, and N. Tatbul, "Neo: A learned query optimizer," *Proc. VLDB Endowment*, vol. 12, no. 11, pp. 1705–1718, 2019.

[13] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, and T. Kraska, "Bao: Making learned query optimization practical," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2021, pp. 1275–1288.

[14] J. Duggan, A. J. Elmore, M. Stonebraker, M. Balazinska, B. Howe, J. Kepner, S. Madden, D. Maier, T. Mattson, and S. Zdonik, "The

BigDAWG polystore system," *ACM SIGMOD Record*, vol. 44, no. 2, pp. 11–16, 2015.

[15] B. Kolev, P. Valduriez, C. Bondiombouy, R. Jiménez-Peris, R. Pau, and J. Pereira, "CloudMdsQL: Querying heterogeneous cloud data stores with a common language," *Distributed and Parallel Databases*, vol. 34, no. 4, pp. 463–503, 2016.

[16] A. Behm, S. Palkar, U. Agarwal, T. Armstrong, D. Cashman, A. Dave, T. Greenstein, S. Hovsepian, et al. "Photon: A fast query engine for lakehouse systems," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2022, pp. 2326–2339.

[17] M. Vuppalapati, J. Miron, R. Agarwal, D. Truong, A. Motivala, and T. Cruanes, "Building an elastic query engine on disaggregated storage," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, pp. 449–462.

[18] Apache Software Foundation, "Apache Arrow: A cross-language development platform for in-memory analytics." [Online]. Available: https://arrow.apache.org

[19] Apache Software Foundation, "Apache Parquet: A columnar storage format for the Hadoop ecosystem." [Online]. Available: https://parquet.apache.org

[20] New York City Taxi and Limousine Commission, "TLC trip record data." [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[21] A. Pavlo, E. Paulson, A. Rasin, D. J. Abadi, D. J. DeWitt, S. Madden, and M. Stonebraker, "A comparison of approaches to large-scale data analysis," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, 2009, pp. 165–178.

[22] F. McSherry, M. Isard, and D. G. Murray, "Scalability! But at what COST?," in *Proc. 15th Workshop on Hot Topics in Operating Systems (HotOS)*, 2015.

APPENDIX: ARTIFACTS AND REPRODUCIBILITY

The submission contains the full implementation as a single Jupyter notebook that runs end to end under Restart Kernel → Run All, with dependencies pinned in requirements.txt and interactive input disabled. Raw data is fetched idempotently from the public TLC endpoint [20] and is therefore not redistributed. Every number, table and figure in this paper is produced by that notebook and exported to the results directory as CSV - one file per table, plus the figure sources. No value reported here was entered by hand.